Calendly Marketing Insights

Solution Design Document

This document is intended for design approval. The key approval items are the AWS architecture, Bronze/Silver/Gold Delta Lake storage approach, proposed data model, dashboard metrics, and failure/reload strategy.

1. Proposed Solution Overview

The DEA Calendly Marketing Insights capstone will consist of the following design elements:

- Calendly invitee.created webhook events will be sent to a public HTTPS endpoint exposed through Amazon API Gateway. API Gateway will forward POST requests to a Lambda webhook receiver. The Lambda function will validate the request shape, capture the raw webhook body, add ingestion metadata, and write the event to the Bronze layer in S3. This preserves the original event payload for replay, auditing, and downstream reprocessing.
- AWS Lambda will be used because webhook events are small and event-driven.
- Amazon EventBridge will trigger a scheduled Lambda function once per day after the marketing spend files are expected to be available. The spend ingestion Lambda will read the public S3 file index or dated spend file path, perform lightweight validation, and write the raw spend data to the Bronze layer in S3. This creates a repeatable daily ingestion process independent of manual file downloads.
- Glue PySpark will be used because the final pipeline needs scalable transforms and Delta Lake writes.
- S3 is used because the requirement calls for Delta Lake on top of AWS S3.
- The Bronze layer stores raw source data with minimal transformation. The Silver layer stores cleaned, typed, flattened Delta tables for Calendly bookings and marketing spend. The Gold layer stores dashboard-ready metric tables such as daily calls booked by source, cost per booking by channel, booking trends, channel attribution, booking heatmap data, and employee meeting load.
- Streamlit will be used to implement the final dashboard.

2. Business Objective

a. The Calendly Marketing Insights solution must provide business stakeholders with a reliable way to measure marketing campaign effectiveness and meeting scheduling behavior. The organization needs to understand how many leads are booking calls through each marketing channel, how much each channel costs per booked call, and how meeting activity is distributed across campaigns, time slots, and employees.
b. The solution must ingest newly created Calendly invitee events and daily marketing spend records, transform those sources into analytics-ready datasets, and expose dashboard metrics for business review. Calendly events must be filtered to the supported marketing event types for Facebook paid ads, YouTube paid ads, and TikTok paid ads. Marketing spend files must be ingested from the provided public S3 location and associated with booking activity by date and channel.
c. The dashboard must support the following required business insights: daily calls booked by source, cost per booking by channel, booking trends over time, channel attribution using UTM/event type data, booking volume by hour and day of week, and weekly meeting load per employee. These insights will help stakeholders compare lead generation performance across channels, identify cost-efficient acquisition sources, monitor campaign trends, optimize scheduling strategy, and identify potential employee over-scheduling.
d. The underlying data pipeline must be production-oriented. It must use AWS services, store data in Delta Lake format on Amazon S3, support scheduled ingestion and transformation, include encryption for stored data, and provide a reload mechanism by preserving raw source data in the Bronze layer for reprocessing into Silver and Gold layers.

3. Source Data

The Calendly Marketing Insights pipeline will ingest two primary source datasets: Calendly webhook event data and marketing spend data.

Calendly Webhook Events

Calendly will provide new booking activity through webhook notifications. The primary event type for this pipeline is `invitee.created`, which is triggered when a new Calendly invitee booking is created.

Each Calendly webhook event contains a nested JSON payload with details about the invitee, the scheduled event, the Calendly event type, the assigned event member or employee, the meeting start and end time, the invitee timezone, the booking status, and tracking fields such as UTM campaign parameters.

Because the Calendly webhook may include events for the broader organization, the pipeline will filter incoming webhook records to the three marketing event types that are relevant to this project:

Facebook Paid Ads
Calendly event type URI:
`https://api.calendly.com/event_types/d639ecd3-8718-4068-955a-436b10d72c78`

YouTube Paid Ads
Calendly event type URI:
`https://api.calendly.com/event_types/dbb4ec50-38cd-4bcd-bbff-efb7b5a6f098`

TikTok Paid Ads
Calendly event type URI:
`https://api.calendly.com/event_types/bb339e98-7a67-4af2-b584-8dbf95564312`

The raw Calendly webhook events will be stored in the Bronze layer in their original JSON format. This preserves the original source payload for auditing, troubleshooting, and reload processing. The raw webhook events will then be transformed into a Silver booking dataset by flattening the nested JSON structure and extracting the fields needed for analytics.

The Silver Calendly booking dataset will include booking identifiers, invitee details, event type, mapped marketing channel, meeting start and end times, derived booking dates and time slots, UTM tracking fields, and employee assignment information. This dataset will provide the booking-level foundation for downstream Gold metrics and dashboard reporting.

Marketing Spend Data

Marketing spend data will be provided as daily JSON files in a public S3 bucket. The daily spend files follow a date-based naming pattern:

`https://dea-data-bucket.s3.us-east-1.amazonaws.com/calendly_spend_data/spend_data_YYYY-MM-DD.json`

The project also provides a `file_index.json` file that can be used by the ingestion process to identify which spend files are available for processing.

Each marketing spend record contains three core fields: the spend date, the marketing channel, and the spend amount in USD. The expected channels are Facebook paid ads, YouTube paid ads, and TikTok paid ads.

The raw marketing spend files will be stored in the Bronze layer in their original JSON format. The data will then be validated and transformed into a Silver marketing spend dataset. The Silver spend dataset will normalize the field names, validate the channel values, standardize the spend date, and represent the spend amount as a numeric USD value.

The marketing spend data will be used with Calendly booking activity to calculate cost-per-booking, channel attribution, and marketing efficiency metrics. The primary join logic between the two datasets will be based on marketing channel and date where applicable.

Source Data Summary

The Calendly webhook source is event-driven and provides near real-time booking activity. Its primary purpose is to capture new booked calls, channel attribution fields, meeting timestamps, and employee assignment data.

The marketing spend source is batch-oriented and arrives daily as JSON files in a public S3 location. Its primary purpose is to provide daily spend by marketing channel so the pipeline can calculate cost-per-booking and channel efficiency.

Both sources will be preserved in the Bronze layer before being cleaned and normalized into Silver datasets. The Silver datasets will then be transformed into Gold-layer metric tables that support the Streamlit dashboard.

4. Proposed Architecture

The proposed solution will use an AWS-native lakehouse architecture built on Amazon S3, Delta Lake, AWS Lambda, Amazon API Gateway, Amazon EventBridge, AWS Glue PySpark, and Streamlit.

Calendly `invitee.created` events will be received through an API Gateway webhook endpoint and processed by a Lambda function. The Lambda function will capture each raw webhook payload and write it to the Bronze layer in Amazon S3.

Marketing spend data will be ingested daily using an EventBridge-triggered Lambda function. The function will read the available JSON spend files from the provided public S3 source and write the raw spend records to the Bronze layer.

Amazon S3 will serve as the central storage layer for the Bronze, Silver, and Gold data zones. Delta Lake will be used as the table format on top of S3 to support reliable table metadata, schema enforcement, incremental processing, and reload/recovery workflows.

AWS Glue PySpark jobs will transform the raw Bronze data into cleaned Silver datasets. The Silver Calendly bookings table will contain flattened booking, campaign, meeting, and employee fields. The Silver marketing spend table will contain validated spend records by date and channel. Additional Glue PySpark jobs will transform the Silver datasets into Gold metric tables for dashboard reporting.

The Gold layer will feed a Streamlit dashboard that provides business insights into marketing campaign performance, cost per booking, booking trends, channel attribution, booking time patterns, and employee meeting load. The Gold tables may be queried through Athena using Glue Data Catalog metadata before being consumed by the Streamlit dashboard.

The design includes scheduled processing, encrypted storage, and reload support. Raw Bronze data will be retained so that Silver and Gold datasets can be regenerated if downstream processing fails or metric logic changes.

5. Data Flow

The pipeline will process two inbound data streams: Calendly webhook events and daily marketing spend files. Both sources will be landed in the Bronze layer before being transformed into Silver and Gold datasets.

Calendly booking data will enter the system through the API Gateway webhook endpoint. When a new Calendly `invitee.created` event occurs, Calendly will send the event payload to API Gateway. API Gateway will forward the request to the webhook ingestion Lambda function. The Lambda function will validate the event structure, add ingestion metadata such as source system, ingestion timestamp, and raw file path, and then write the raw JSON event to the Bronze layer in Amazon S3.

Marketing spend data will follow a scheduled batch flow. An EventBridge rule will trigger a spend ingestion Lambda function once per day after the expected file arrival time. The Lambda function will use the provided public S3 spend file location or file index to identify available spend files. It will retrieve the JSON spend data and write the raw records to the Bronze layer in Amazon S3.

After raw data is stored in Bronze, AWS Glue PySpark jobs will transform the data into Silver Delta tables. The Calendly Silver transformation will flatten the nested webhook JSON and extract booking-level fields such as booking ID, event type, mapped marketing channel, invitee details, meeting start and end time, UTM fields, employee assignment, booking date, meeting hour, day of week, and week start date. The marketing spend Silver transformation will validate the spend records, standardize the

date and channel fields, convert spend values into numeric USD amounts, and preserve source file metadata.

The Gold transformation layer will use the Silver Calendly booking and Silver marketing spend tables to produce dashboard-ready metric tables. These Gold tables will calculate business metrics such as daily calls booked by source, cost per booking by channel, booking trends over time, channel attribution, booking volume by time slot and day of week, and employee meeting load.

The Streamlit dashboard will read from the Gold layer and present the final business insights to stakeholders. The dashboard will not perform raw data processing. Its role will be to visualize prepared Gold metrics through KPI tiles, charts, and tables.

Raw Bronze data will be retained so that the pipeline can reload downstream layers when needed. If a Silver or Gold transformation fails, or if business rules change, the affected tables can be rebuilt from the preserved Bronze data without requiring the source systems to resend historical records.

Data Flow Summary

1. Calendly sends `invitee.created` webhook events to API Gateway.
2. API Gateway invokes the Calendly webhook Lambda function.
3. The Lambda function writes raw Calendly JSON to the Bronze layer in S3.
4. EventBridge triggers the marketing spend ingestion Lambda on a daily schedule.
5. The spend ingestion Lambda reads available spend files from the public S3 source.
6. The spend ingestion Lambda writes raw spend JSON to the Bronze layer in S3.
7. AWS Glue PySpark transforms Bronze data into cleaned Silver Delta tables.
8. AWS Glue PySpark transforms Silver data into Gold metric tables.
9. Streamlit reads Gold metrics and displays the dashboard.
10. Bronze data remains available for failure recovery and reload processing.

6. Bronze / Silver / Gold Layer Design

The pipeline will use a Bronze, Silver, and Gold lakehouse design to separate raw source ingestion, cleaned analytical data, and dashboard-ready business metrics. This layered approach improves data quality, supports replay and reload processing, and keeps the dashboard isolated from raw source complexity.

Bronze Layer

The Bronze layer will store raw source data exactly as it is received from the upstream systems, with minimal transformation. This layer provides an immutable landing zone for source records and supports auditing, troubleshooting, and reload processing.

Calendly webhook events will be stored in Bronze as raw JSON payloads. Each record will preserve the original webhook body and include ingestion metadata such as ingestion timestamp, source system, event type, and raw S3 object path.

Marketing spend files will also be stored in Bronze as raw JSON. The Bronze spend data will preserve the source file contents and include metadata such as source file name, ingestion timestamp, and processing date.

The Bronze layer will be the recovery point for the pipeline. If a downstream Silver or Gold transformation fails, or if business logic changes, the data can be reprocessed from Bronze without requiring Calendly or the public spend source to resend historical data.

Silver Layer

The Silver layer will contain cleaned, validated, flattened, and typed datasets derived from the Bronze layer. This layer represents the analytics-ready version of the source data.

The Silver Calendly bookings table will flatten the nested Calendly webhook JSON into one booking-level record per invitee booking. It will extract and standardize fields such as booking ID, scheduled event ID, event type URI, mapped marketing channel, invitee details, meeting start and end timestamps, booking date, meeting date, meeting hour, meeting day of week, week start date, UTM tracking fields, and employee assignment information.

The Silver marketing spend table will contain one standardized spend record per date and channel. It will validate that each channel belongs to the supported campaign channels, standardize the spend date, convert spend into a numeric USD value, and preserve source file metadata for traceability.

The Silver layer will provide the primary joinable datasets used by the Gold layer. Calendly booking data and marketing spend data will be associated primarily by channel and date where applicable.

Gold Layer

The Gold layer will contain curated, dashboard-ready metric tables built from the Silver datasets. These tables will be structured around the business questions required for the final Streamlit dashboard.

Gold tables will include metrics for daily calls booked by source, cost per booking by channel, booking trends over time, channel attribution, booking volume by time slot and day of week, and meeting load per employee.

The Gold layer will be optimized for reporting and visualization. Instead of requiring the dashboard to process raw or deeply nested data, the dashboard will read prepared metric outputs from the Gold layer. This keeps the dashboard simpler, faster, and more reliable.

Layer Summary

- Bronze stores raw source data and ingestion metadata.
- Silver stores cleaned, validated, flattened, and typed analytical records.
- Gold stores aggregated business metrics prepared for dashboard reporting.

This design supports both operational reliability and analytical clarity. Raw data is preserved for reloads, Silver data provides reusable normalized datasets, and Gold data provides business-ready outputs for stakeholders.

7. Delta Lake Storage Strategy

The solution will use Delta Lake format on top of Amazon S3 for the analytical lakehouse layers. Delta Lake will allow the pipeline to store Parquet-based tables with transaction logs, schema enforcement, incremental updates, and table version history.

In this design, Delta Lake is implemented as a table format stored in S3, not as a separate AWS service. Each Delta table will be represented by an S3 folder containing Parquet data files and a `_delta_log` directory that tracks changes to the table.

The S3 data lake will be organized into Bronze, Silver, and Gold zones. Bronze will preserve raw Calendly webhook events and raw marketing spend data. Silver will store cleaned and normalized Delta tables for Calendly bookings and marketing spend. Gold will store aggregated Delta tables that support the final dashboard metrics.

AWS Glue PySpark jobs will read from each layer and write the transformed output to the next layer in Delta format. This approach supports reliable processing, schema consistency, incremental loads, and reload recovery from preserved Bronze data.

The proposed storage structure will include separate S3 prefixes for each layer and table, such as Bronze Calendly webhooks, Bronze marketing spend, Silver Calendly bookings, Silver marketing spend, and Gold metric tables for cost per booking, channel attribution, booking trends, time-slot analysis, and employee meeting load.

This strategy meets the requirement to store the pipeline data in Delta Lake format on AWS S3 while supporting a production-oriented Bronze/Silver/Gold lakehouse design.

8. Transformation Logic

The transformation layer will convert raw source data from the Bronze layer into cleaned Silver datasets and dashboard-ready Gold metrics. AWS Glue PySpark jobs will perform the production transformations using Delta Lake tables stored on Amazon S3.

Bronze to Silver Transformations

The first transformation stage will convert raw Bronze data into normalized Silver tables.

For Calendly webhook data, the Bronze to Silver transformation will read raw `invitee.created` webhook JSON records from the Bronze Calendly location. The job will flatten the nested Calendly payload into one booking-level row per invitee booking. It will extract booking identifiers, scheduled event identifiers, event type URI, invitee details, meeting timestamps, booking status, UTM tracking fields, employee assignment fields, and location details.

The Calendly transformation will also map the Calendly event type URI to the appropriate marketing channel. The supported channels are Facebook paid ads, YouTube paid ads, and TikTok paid ads. Records that do not match the supported event types may be excluded from the marketing analytics tables or retained separately for audit purposes.

The Calendly transformation will derive additional date and time fields needed for downstream metrics. These fields include booking date, meeting date, meeting hour, meeting day of week, and meeting week start date. These derived fields will support trend reporting, time-slot analysis, and employee meeting load calculations.

For marketing spend data, the Bronze to Silver transformation will read raw spend JSON records from the Bronze marketing spend location. The job will validate that each record contains a spend date, supported marketing channel, and numeric spend value. It will standardize field names, convert spend amounts to numeric USD values, and preserve source file metadata for traceability.

The output of this stage will be two primary Silver Delta tables: Silver Calendly Bookings and Silver Marketing Spend.

Silver to Gold Transformations

The second transformation stage will convert Silver datasets into Gold metric tables for reporting.

The Silver Calendly Bookings table will provide booking activity, channel attribution, meeting timestamps, and employee assignment data. The Silver Marketing Spend table will provide date and channel-level spend amounts. These two Silver datasets will be joined or aggregated by channel and date where needed.

The Gold transformation will calculate the required dashboard metrics. Daily calls booked by source will be calculated by counting distinct booking IDs by booking date and channel. Cost per booking by channel will be calculated by dividing total spend by total booked calls for each channel. Booking trends will be calculated by aggregating booking counts by date and channel. Channel attribution will use mapped event types and UTM fields where available to group bookings by channel and campaign. Booking volume by time slot will use derived meeting hour and day-of-week fields. Employee meeting load will group meetings by employee and week.

The Gold layer will store each major metric output as a separate Delta table. This keeps the Streamlit dashboard simple and efficient because the dashboard can read prepared reporting tables instead of performing complex transformations at runtime.

Data Quality and Validation

The transformation jobs will include basic validation logic to improve data quality. Calendly records must include a supported webhook event type, a non-empty payload, a scheduled event, and a supported event type URI for marketing attribution. Marketing spend records must include a valid date, supported channel, and non-negative numeric spend value.

Invalid or unsupported records should not break the entire pipeline. Depending on implementation needs, they may be skipped, logged, or written to a separate error location for review. This allows the pipeline to continue processing valid records while preserving information about rejected records.

Incremental Processing and Reloads

The transformation design will support incremental processing where possible. New Calendly webhook records can be appended to the Bronze layer as they arrive, and daily spend files can be processed as they become available. Glue PySpark jobs can then process new or changed data into the Silver and Gold Delta tables.

Because raw source data is preserved in the Bronze layer, Silver and Gold tables can be rebuilt if a transformation fails or if business logic changes. This reload capability is important for operational recovery and for supporting future updates to attribution or metric definitions.

Transformation Output

The final transformation outputs will be Gold Delta tables that support the Streamlit dashboard. These tables will provide prepared business metrics for marketing performance, cost efficiency, booking trends, time-slot analysis, and employee meeting load.

This transformation approach satisfies the project requirement to extract, store, transform, and provide business insights from Calendly and marketing data using a production-oriented AWS pipeline. The required business metrics include daily calls booked by source, cost per booking, booking trends, channel attribution, booking volume by time slot, and employee meeting load.

9. Metrics and Dashboard Outputs

The final Streamlit dashboard will present Gold-layer metrics that help business stakeholders evaluate marketing performance, booking activity, scheduling patterns, and employee meeting load. The dashboard will read prepared metric tables from the Gold layer rather than performing raw data transformations at runtime.

The dashboard will include top-level KPI tiles that summarize total bookings, total marketing spend, and average cost per booking across the selected reporting period. These KPIs will give stakeholders a quick view of overall campaign performance.

The Daily Calls Booked by Source view will show how many Calendly bookings were created by source or marketing channel each day. This output will help stakeholders monitor daily lead generation activity and compare booking volume across Facebook paid ads, YouTube paid ads, and TikTok paid ads.

The Cost Per Booking by Channel view will calculate the average cost required to generate a booked call for each marketing channel. The calculation will use the formula: Cost Per Booking equals Total Spend divided by Total Booked Calls. This output will help identify which channels are generating booked calls most efficiently.

The Bookings Trend Over Time view will show daily or weekly booking volume by marketing source. This output will help stakeholders identify changes in booking activity over time, including campaign surges, declines, fatigue, seasonality, or the impact of marketing changes.

The Channel Attribution view will attribute bookings to marketing channels and campaigns using the mapped Calendly event type and UTM fields where available. This output will show total bookings, total spend, and cost per booking by channel and

campaign. It will support leaderboard-style analysis of which sources are producing the highest volume and most cost-effective booked calls.

The Booking Volume by Time Slot and Day of Week view will show when leads are most likely to schedule calls. This output will use the meeting timestamp to group bookings by hour of day and day of week. The result can help inform ad scheduling, sales team coverage, and calendar availability planning.

The Employee Meeting Load view will show how many meetings each employee attends by week. This output will help stakeholders understand workload distribution and identify potential over-scheduling or uneven meeting assignment patterns.

The dashboard will use visualizations such as KPI tiles, line charts, bar charts, tables, and heatmap-style views. The Gold tables will be designed so that each dashboard section can read directly from a prepared metric output, keeping the dashboard simple, maintainable, and focused on business interpretation.

The required dashboard metrics include daily calls booked by source, cost per booking by channel, booking trends over time, channel attribution, booking volume by time slot and day of week, and meeting load per employee.

10. Scheduling

The pipeline will use a combination of event-driven ingestion and scheduled batch processing.

Calendly webhook ingestion will be event-driven. When a new Calendly `invitee.created` event occurs, Calendly will send the webhook payload to the API Gateway endpoint. API Gateway will invoke the Calendly webhook Lambda function immediately, allowing new booking events to be captured in near real time. Because this process is triggered by incoming webhook events, it does not require a scheduled job.

Marketing spend ingestion will use a scheduled batch process. The project requirements indicate that marketing spend files will be available daily at approximately 06:00 AM EST for the prior day's spend data. An Amazon EventBridge rule will trigger the marketing spend ingestion Lambda once per day after the expected file availability time. The Lambda function will check the provided public S3 location or file index, identify available spend files, and ingest the appropriate records into the Bronze layer.

Transformation jobs will also be scheduled. After the daily spend ingestion process completes, AWS Glue PySpark jobs will transform new Bronze data into Silver Delta tables. A subsequent Glue job will transform Silver data into Gold metric tables for dashboard reporting. These transformation jobs may be triggered by EventBridge on a daily schedule or chained after successful completion of the ingestion process.

The proposed daily schedule is:

- Calendly webhook ingestion: event-driven, triggered whenever Calendly sends a new booking event.
- Marketing spend ingestion: daily, after the expected 06:00 AM EST spend file availability window.
- Bronze to Silver transformations: daily, after spend ingestion completes, and/or on a recurring schedule for accumulated webhook data.
- Silver to Gold metric transformations: daily, after Silver transformations complete.
- Streamlit dashboard refresh: reads from the latest Gold outputs. The dashboard can be refreshed on demand by users or configured to reload data at runtime.

This scheduling design supports both near real-time booking capture and daily marketing spend processing. It also ensures that Gold-layer dashboard metrics are refreshed after the required source data has been ingested and transformed. Since raw data is preserved in Bronze, scheduled jobs can be rerun if processing fails or if a reload is needed.

11. Security and Encryption

The pipeline will use AWS security controls to protect stored data, limit access to pipeline resources, and reduce unnecessary exposure of source data.

All data stored in Amazon S3 will use server-side encryption. The S3 buckets used for Bronze, Silver, and Gold data will be configured with default encryption, using either Amazon S3-managed encryption keys or AWS KMS-managed keys depending on implementation requirements. This ensures that raw webhook events, marketing spend data, Delta tables, and dashboard-ready outputs are encrypted at rest.

Access to S3 data will be controlled through IAM policies using the principle of least privilege. Lambda functions and AWS Glue jobs will only be granted access to the specific buckets and prefixes needed for their role. For example, the Calendly webhook Lambda will be allowed to write raw Calendly events to the Bronze Calendly prefix, while Glue transformation jobs will be allowed to read from Bronze and write to Silver or Gold as needed.

API Gateway will expose the public HTTPS endpoint required for Calendly webhook delivery. The Lambda webhook receiver will validate incoming event structure before writing data to S3. If Calendly webhook signing or a shared secret is available, the implementation can validate the webhook signature or secret to reduce the risk of accepting unauthorized webhook requests.

Sensitive configuration values, such as API tokens, webhook secrets, or environment-specific settings, will not be hardcoded in source code. These values will be stored in AWS Secrets Manager, AWS Systems Manager Parameter Store, or Lambda

environment variables as appropriate. Source code and configuration files committed to GitHub will avoid storing secrets or real personal data.

AWS CloudWatch Logs will be used for operational logging and troubleshooting. Logs should capture useful processing information such as ingestion status, file names, record counts, and error messages. Logs should avoid printing full raw webhook payloads or unnecessary personally identifiable information.

Network exposure will be limited to the required public API Gateway endpoint. Internal processing components such as Lambda functions, Glue jobs, and S3 buckets will use IAM-based access controls rather than public access. S3 public access should remain blocked for project-owned buckets unless a specific exception is required.

The Streamlit dashboard will read from prepared Gold-layer outputs rather than raw Bronze data. This limits dashboard exposure to curated business metrics and avoids unnecessary access to raw webhook payloads.

This security approach provides encrypted storage, controlled access, secure configuration handling, and reduced exposure of raw source data while still supporting the required ingestion, transformation, and reporting workflow.

12. Failure Handling and Reload Strategy

The pipeline will be designed so that source data is preserved before downstream transformations occur. This allows failed jobs to be retried and downstream tables to be rebuilt without requiring the original source systems to resend historical data.

The Bronze layer will serve as the primary recovery point for the pipeline. Raw Calendly webhook events and raw marketing spend files will be written to S3 before they are transformed into Silver or Gold datasets. Because Bronze data is retained, the pipeline can replay source data if a Silver or Gold transformation fails or if business logic needs to be changed later.

For Calendly webhook ingestion, the Lambda function will write each valid incoming event to the Bronze layer with ingestion metadata such as event type, ingestion timestamp, and S3 object path. If a webhook event cannot be processed due to invalid structure or missing required fields, the error will be logged in CloudWatch. Depending on implementation needs, invalid records may also be written to a separate error prefix in S3 for later review.

For marketing spend ingestion, the scheduled Lambda function will check for available spend files and write raw spend records to Bronze. If the expected file is not available, the Lambda function will log the missing file condition and exit safely rather than failing

downstream transformations. The job can be rerun after the file becomes available. If a spend file is malformed or contains invalid records, those records can be logged or written to an error location while valid records continue through the pipeline.

AWS Glue PySpark transformation jobs will be designed to be rerunnable. Bronze-to-Silver jobs can regenerate the Silver Calendly bookings and Silver marketing spend Delta tables from preserved Bronze data. Silver-to-Gold jobs can regenerate dashboard metric tables from the Silver datasets. This provides a reload path for both operational failures and future changes to metric definitions.

Delta Lake supports reliable table writes through transaction logs. If a job fails during a write operation, the Delta transaction log helps prevent partially committed table states from being treated as valid output. This improves reliability compared with writing unmanaged files directly to S3.

CloudWatch Logs will be used to capture job status, processing counts, and error details for Lambda and Glue jobs. Failed Lambda executions and failed Glue jobs can be reviewed in CloudWatch to identify the failure point. After the issue is corrected, the affected ingestion or transformation job can be rerun.

The proposed reload strategy is:

1. Preserve all raw source data in the Bronze layer.
2. Log invalid or unsupported records without discarding the entire batch.
3. Make ingestion and transformation jobs rerunnable.
4. Rebuild Silver tables from Bronze when source parsing or validation logic changes.
5. Rebuild Gold tables from Silver when metric logic or dashboard requirements change.
6. Use Delta Lake transaction logs to reduce the risk of partially written tables.
7. Use CloudWatch logging to identify failed jobs and support troubleshooting.

This failure handling and reload strategy ensures that the pipeline can recover from missing files, malformed records, failed transformations, and future business logic changes while preserving the original source data needed for reprocessing.

13. CI/CD Approach

The project will use GitHub as the source code repository and GitHub Actions for continuous integration and deployment support. The CI/CD process will help validate code changes, reduce manual errors, and provide a repeatable path for packaging and deploying pipeline components.

The initial CI workflow will run automatically when changes are pushed to the repository or when a pull request is opened. This workflow will set up the required Python version, install dependencies from `requirements.txt`, and run the project's automated test suite using `pytest`. These tests will validate the parser logic, transformation logic, and metric calculations before code changes are merged or deployed.

The CI pipeline will also support basic code quality checks. Depending on project scope, this may include linting, formatting checks, or import validation. The goal is to catch common errors early, such as broken imports, invalid parser behavior, or incorrect metric calculations.

For deployment, the pipeline can be extended to package AWS Lambda functions and deploy updated code to AWS. Lambda deployment artifacts can be built from the repository and uploaded through AWS CLI commands or an infrastructure deployment process. AWS credentials used by GitHub Actions should be stored securely as GitHub repository secrets and should follow least-privilege access.

AWS Glue job scripts will also be version-controlled in the repository. Deployment of Glue scripts can be automated by uploading the latest approved scripts to the appropriate S3 scripts location and updating Glue job configuration where needed. This ensures that transformation logic is traceable and reproducible.

The CI/CD process will follow a staged approach:

1. Validate code locally using `pytest`.
2. Commit changes to GitHub.
3. GitHub Actions installs dependencies and runs automated tests.
4. If tests pass, code is considered valid for deployment.
5. Lambda and Glue scripts can be packaged and deployed to AWS.
6. Deployment actions use secured credentials and avoid hardcoded secrets.

This approach supports the project requirement to include a CI/CD pipeline using GitHub while keeping the implementation lightweight and appropriate for the capstone. The first version of CI/CD will focus on automated validation, with deployment automation added for Lambda and Glue components as the AWS implementation is completed.

14. Assumptions and Open Questions

This solution design is based on the requirements provided for the Calendly Marketing Insights project and on the sample webhook and marketing spend data currently available. The following assumptions and open questions should be confirmed with the SME before implementation proceeds.

Assumptions

1. The Calendly webhook will send `invitee.created` events to the API Gateway endpoint created for this project.
2. The webhook payload structure will be consistent with the sample Calendly `invitee.created` JSON provided in the requirements document.
3. Only the three provided Calendly event type URIs are in scope for marketing campaign analysis: Facebook paid ads, YouTube paid ads, and TikTok paid ads.
4. The Calendly event type URI can be used as the primary mapping field for marketing channel attribution.
5. UTM fields may be null in some webhook events. When UTM fields are not populated, channel attribution will rely on the Calendly event type mapping.
6. Marketing spend files will be available daily in the provided public S3 location after the expected file availability time.
7. The marketing spend data will contain date, channel, and spend fields, with spend values represented in USD.
8. Marketing spend is provided at the date and channel level. If campaign-level spend is not available, campaign-level cost metrics will be limited to the available channel-level spend data.
9. The pipeline will use AWS-native services and Delta Lake format on Amazon S3 rather than Databricks, unless the SME requests the Databricks architecture option.
10. The Streamlit dashboard will read prepared Gold-layer metric outputs rather than performing raw transformations at runtime.
11. Raw Bronze data will be retained to support replay, reloads, and future changes to transformation or metric logic.

Open Questions

- Should non-marketing Calendly event types be discarded, stored only in Bronze, or written to a separate rejected/unsupported records location?
- Should canceled or rescheduled bookings be included in booking counts, or should only active bookings count toward final metrics?
- If an invitee reschedules a meeting, should the pipeline update the existing booking record, create a new booking record, or track both the original and rescheduled booking history?
- Should cost per booking be calculated using booking creation date, meeting date, or both?
- Should marketing spend be joined to bookings strictly by date and channel, or should some metrics aggregate spend and bookings across a broader reporting period?
- How should campaign-level attribution be handled when UTM campaign fields are null but the event type maps to a known paid channel?

- Should employee meeting load be calculated based on meeting start date, booking creation date, or both?
- Should the dashboard display data in UTC, the invitee timezone, or a business-standard timezone such as Eastern Time?
- What level of historical backfill is expected for marketing spend files and Calendly booking events?
- Should invalid records be skipped and logged, or should they be written to a dedicated error table or S3 error prefix for review?
- Should S3 encryption use default S3-managed keys or a customer-managed AWS KMS key?
- Will the SME provide a Calendly webhook secret or signature validation mechanism for webhook authentication?
- Should the final dashboard be hosted locally for review, deployed to Streamlit Community Cloud, or deployed using an AWS-hosted option?
- Are there any required naming conventions for AWS resources, S3 prefixes, Glue jobs, or dashboard files?

15. SME Approval Request

Please review the proposed solution design for the Calendly Marketing Insights pipeline. This document describes the planned AWS architecture, source data ingestion, Bronze/Silver/Gold lakehouse design, Delta Lake storage strategy, data model, transformation logic, dashboard metrics, scheduling, security, failure handling, and CI/CD approach.

Approval is requested before proceeding with implementation. Once approval is received, the project will move forward with building the AWS ingestion pipeline, Delta Lake tables, Glue PySpark transformations, and Streamlit dashboard based on this design.

SME Sign-Off

Reviewer Name:

Reviewer Role:

Approval Decision: Approved / Approved with Changes / Not Approved

Comments:

Date:

Written Approval or Signature: